

MANUALE UTENTE E TECNICO: CITYLOGIC

1. Introduzione e Metriche Chiave

Citylogic è un simulatore urbano gestionale *rule-based* incentrato sulla pianificazione strategica e sulla gestione della crescita metropolitana su una griglia logica fissa di 24x16 celle. L'obiettivo dell'amministratore è ottimizzare in modo sinergico 7 metriche chiave che determinano lo stato di salute dell'insediamento:

- **Finanze:** Il bilancio economico e la cassa municipale.
- **Popolazione:** Il numero totale di abitanti residenti.
- **Felicità:** Il livello di soddisfazione globale dei cittadini.
- **Lavoro:** Il tasso di occupazione generale.
- **Sicurezza:** La protezione dal crimine e dalle emergenze.
- **Sanità:** L'efficienza del sistema di cura medica.
- **Ecologia:** L'impatto ambientale delle scelte urbanistiche.

L'avanzamento della simulazione è regolato da un motore a eventi discreti cadenzato da unità temporali denominate **Tick**. Ad ogni scatto del Tick, il sistema esegue il ricalcolo completo di tutte le business rules, aggiornando le risorse e applicando i bilanci derivanti dalla disposizione delle strutture sulla griglia.

2. Configurazione di Ambiente e Avvio

Il software è interamente sviluppato utilizzando la piattaforma Java. Per la compilazione del codice sorgente e l'esecuzione dell'applicazione sono richiesti i seguenti requisiti tecnici:

- **Java Development Kit (JDK):** Versione 17 o superiore.
- **Apache Maven:** Strumento di build automation configurato nel PATH di sistema.

Procedura di lancio:

1. Aprire il terminale nella directory radice del progetto.
2. Eseguire il comando `mvn clean install` per risolvere le dipendenze e compilare i sorgenti.
3. Avviare l'applicazione digitando: `mvn javafx:run`

3. Interfaccia Utente e Pannelli di Controllo

L'architettura grafica è realizzata tramite JavaFX ed è suddivisa in specifiche aree funzionali:

- **Mappa Centrale (Urban Grid):** Area interattiva 24x16 in cui ogni cella costituisce un'unità atomica che può ospitare una singola strada o un singolo edificio.
- **TopBar (Barra di Stato Superiore):** Mostra in tempo reale l'andamento quantitativo dei parametri vitali: Fondi disponibili, Popolazione totale, tick corrente, Felicità, Sicurezza, Salute, Lavoro ed Ecologia.
- **SideBar (Strumenti di Costruzione):** Raccoglie i selettori per l'edificazione di Zone (Residenziali, Commerciali, Industriali), Servizi Pubblici (Ospedali, Polizia, Pompieri) e Infrastrutture Stradali.
- **Pannelli di Destra (Reti Idriche ed Elettriche):** Sezione dedicata al monitoraggio delle risorse infrastrutturali. Visualizza la capacità erogata e il carico attuale della rete elettrica (Centrali) e idrica (Impianti),

- **Pannello Controlli Tempo:** Permette l'avanzamento di un solo tick oppure l'avanzamento automatico (in velocità 1x, 2x o 4x) e l'interruzione del flusso temporale (Pausa).
- **Pannello Notifiche:** Visualizzatore di messaggi di sistema a scorrimento. Genera avvisi in tempo reale esclusivamente per segnalare anomalie critiche (es. "Fondi insufficienti") o per notificare l'attivazione di allarmi legati a disastri imminenti.

4. Validazione e Ciclo di Vita delle Celle

Il posizionamento di un'entità sulla mappa è regolato dal componente **BuilderValidator**. Una cella della griglia logica può transitare esclusivamente tra tre stati funzionali ben definiti:

A. Stato EMPTY (Vuota)

La cella non contiene alcuna struttura ed è disponibile per ospitare un nuovo elemento.

B. Stato DEVELOPING (In Sviluppo / Non Valido)

L'edificio è stato fisicamente posizionato sulla griglia ma non soddisfa i requisiti minimi di validità strutturale per poter funzionare. Una struttura rimane congelata nello stato **DEVELOPING** (senza produrre effetti sulle metriche e senza generare tasse) qualora si verifichi anche una sola delle seguenti condizioni:

1. **Mancanza di Collegamento Stradale:** L'edificio non è direttamente adiacente a un tratto di strada valido.
2. **Mancanza di Risorse Essenziali:** La cella non si trova all'interno del raggio di copertura geografica di una Centrale Elettrica e di un Impianto Idrico funzionanti.
3. **Violazione dei Vincoli sugli Edifici Statali:** L'edificio viene considerato non valido se non presenta una copertura di sicurezza e assistenza coordinata. Nello specifico, l'edificio deve disporre obbligatoriamente di una stazione dei Pompieri, di un posto di Polizia e di un Ospedale situati entro un raggio d'azione minimo di 7 celle di distanza.

C. Stato ACTIVE (Attivo)

L'edificio soddisfa tutti i vincoli del **BuilderValidator**. In questo stato la struttura diventa operativa, partecipa al calcolo economico del Tick e propaga i propri effetti ambientali e sociali.

5. Motore di Simulazione e Formule Logiche (Business Rules)

Ad ogni Tick, il **Simulation Engine** esegue il ricalcolo matematico dei parametri della città secondo formule di dominio ben definite:

- **Entrate:** $(ZoneCommerciali * 10 + ZoneIndustriali * 15) * MoltiplicatorePolicy$
- **Ecologia:** $ValoreBase - (ZoneIndustriali * 5) + (Parchi * 3)$
- **Sanità:** $(Ospedali / PopolazioneTotale) * 100 - (Inquinamento * 0.5)$
- **Sicurezza:** $(StazioniPolizia / PopolazioneTotale) * 100 - (ZoneIndustriali * 0.2)$
- **Felicità:** $(Occupazione * 0.5) + (Parchi * 1.2) - (Inquinamento * 2.0) + (Sanità * 0.4) + (Sicurezza * 0.4)$

Come evidenziato dalle equazioni, le metriche di **Sanità** e **Sicurezza** agiscono come modificatori diretti della **Felicità** globale dei cittadini, costringendo l'utente a bilanciare lo sviluppo industriale con la copertura dei servizi di protezione e cura medica.

6. Gestione delle Politiche ed Eventi Casuali

L'utente può alterare le regole di calcolo a runtime tramite l'attivazione delle Politiche Cittadine (strutturate nel codice tramite il pattern *Strategy*). L'attivazione di una specifica

linea d'azione (es. Politica Ambientale o Politica Industriale) sostituisce dinamicamente l'algoritmo di calcolo, modificando i coefficienti associati alla generazione dell'inquinamento o all'efficienza fiscale.

Il sistema include un modulo per il trigger di eventi imprevisti. Gli eventi possono essere di natura macroeconomica (come la Crisi Economica, che riduce temporaneamente il moltiplicatore delle entrate) o catastrofi fisiche, come la **Pioggia di Meteoriti**. Quando si verifica una Pioggia di Meteoriti, gli edifici situati nelle celle colpite vengono parzialmente distrutti e perdono all'istante i requisiti di validità. Non esistendo uno stato permanente di abbandono, le strutture colpite **tornano istantaneamente nello stato DEVELOPING**.

L'utente dovrà ricostruire i collegamenti viari, le utenze o i servizi nel raggio d'azione per consentire all'edificio di transitare nuovamente verso lo stato **ACTIVE**.

7. Architettura Tecnica e Persistenza dei Dati

Il codice sorgente dell'applicazione è ingegnerizzato seguendo il pattern architetturale **Boundary-Control-Entity (BCE)**, garantendo una netta separazione tra lo strato di presentazione (JavaFX), i controllori della logica di business (Simulation Engine, Validators) e le entità di dominio (Celle, Edifici). Sono inoltre implementati i design pattern GoF *Strategy* per la gestione delle politiche, *Factory* per l'istanziamento delle entità urbane e *Observer* per notificare alla UI le variazioni di stato dei parametri al termine di ogni Tick.

Il sistema di salvataggio e caricamento è affidato alla classe **PersistenceManager**. La persistenza dei dati di gioco avviene tramite l'accesso diretto al FileSystem locale mediante la generazione di file in formato standard **JSON**, gestiti internamente attraverso l'ausilio delle librerie di serializzazione Jackson/Gson.

Lo snapshot memorizzato include la configurazione topologica completa della Urban Grid, lo stato corrente di transizione di ciascuna cella, i fondi residui all'interno della cassa comunale e la cronologia dei tick. Al riavvio dell'applicazione, il caricamento del file JSON ricostruisce lo stato delle entità con una fedeltà pari al 100%. L'integrità dei flussi di calcolo e l'assenza di regressioni sono verificate in fase di build tramite una suite di test automatizzati scritti in JUnit 5.